Advanced ASIC Design

Exercise no 5: Fusion Compiler—synthesis and top-down implementation

Krzysztof Dziembała

2025-06-25

Contents

1 Floorplan creation **3**

2 Cell placement **6**

3 Clock tree synthesis **12**
3.1 Task 3 script 17

4 Routing **25**

5 Signoff **28**

List of Figures

1.1 Automatically created layout. . . . 3
1.2 Manually created layout. . . . 4
1.3 Final layout with power network. . . . 4
1.4 Final layout with power network. Power network is hidden, except for cell power. . . . 5

2.1 Layout (power network hidden) after `classic_placement` stage. . . . 7
2.2 Layout (power network hidden) after `unified_placement` stage. . . . 7
2.3 Layout (power network hidden) after `placement_with_blockage` stage. 8
2.4 10 cells with largest displacements during legalisation and their displacement vectors. . . . 8
2.5 All cells displaced during legalisation, colour coded according to their displacement. . . . 9
2.6 Max delay path of final `placement_and_optimization` block plotted on layout after reopening the design. . . . 11

3.1 Clock routing after `build_clock` phase. . . . 16
3.2 Clock routing after `route_clock` phase. . . . 18
3.3 Clock routing after `final_opto` phase, without global routes. . . . 18
3.4 Clock routing after `final_opto` phase, with standard cells visible and global routes hidden. . . . 19
3.5 Clock routing and ground shields. Rings, straps, standard cells and their power are hidden. . . . 19
3.6 Ground shields only, with clock hidden. . . . 20
3.7 Highlighted clocks tree at the end of the task. . . . 20

4.1 Layout view after routing. . . . 25
4.2 Layout view after routing with power and ground routes hidden. . . . 26

5.1 Final layout. . . . 28
5.2 Final layout with power and ground routes hidden. . . . 30

Task 1. Floorplan creation

- `core_utilization` is 0.65 (defined as variable `CoreUtilization`); `core_offset` is 5 (defined as variable `CoreOffset`)
- width to height core ratio is 5:3 (defined by `CoreSideRatio` 2.5 1.5)
- I/O pins are placed on left, top and right edges
- clock pin is placed on bottom edge
- I/O pins are assigned to layers M3 and M4
- clock pin is assigned to layer M5
- standard cell power and ground rails use M1 layer and run horizontally, even though the layer routing direction is defined as vertical
- power and ground rings use layers M6 (horizontal) and M7 (vertical)
- power straps use layers M5 (vertical), M6 (horizontal) and M7 (vertical)
- ground straps use layers M5 (vertical), M6 (horizontal) and M7 (vertical)

Figure 1.1: Automatically created layout.

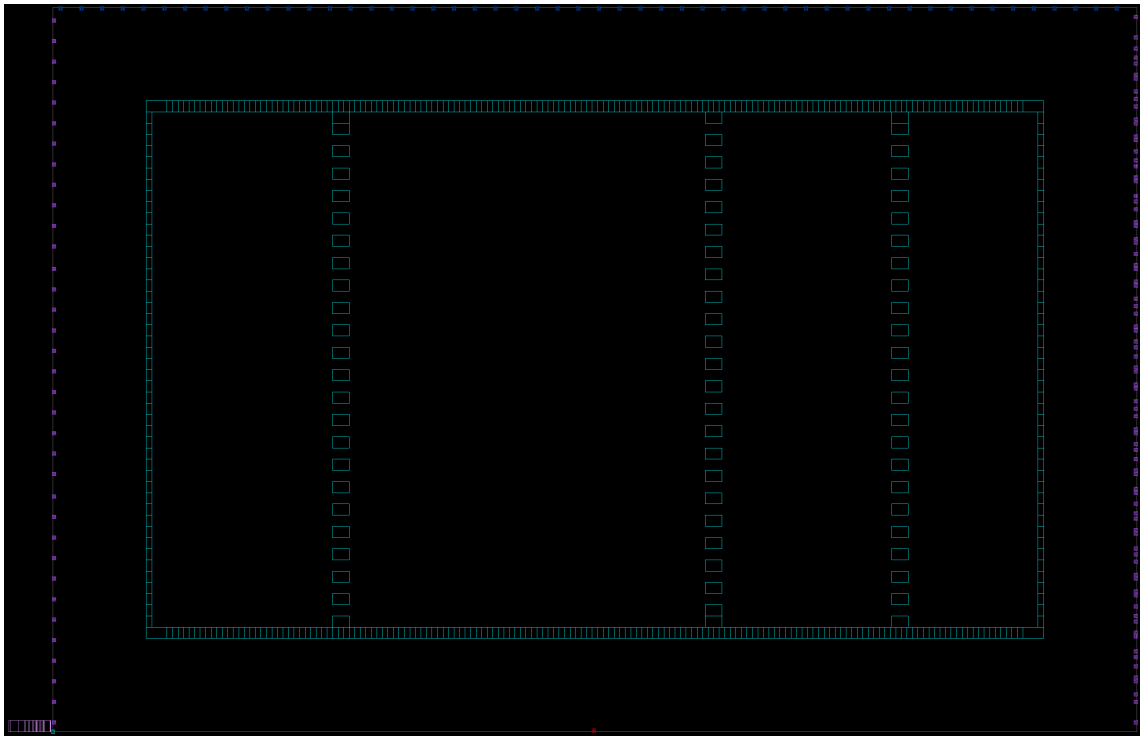

Figure 1.2: Manually created layout.

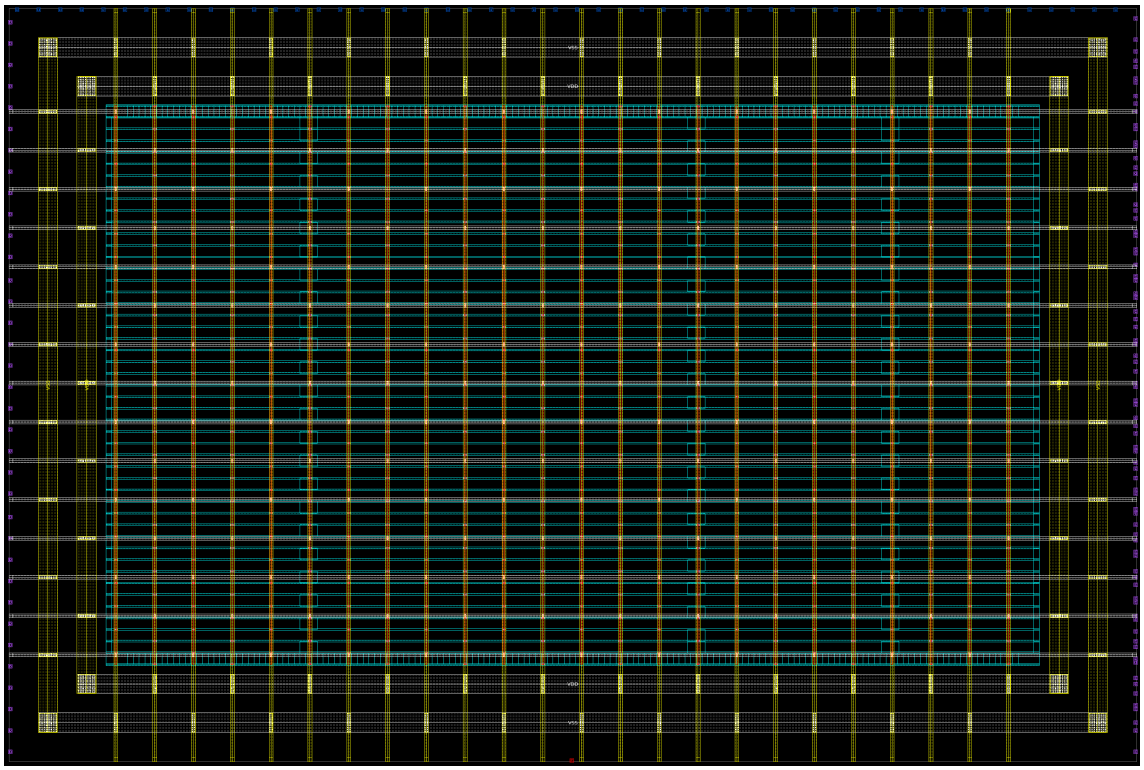

Figure 1.3: Final layout with power network.

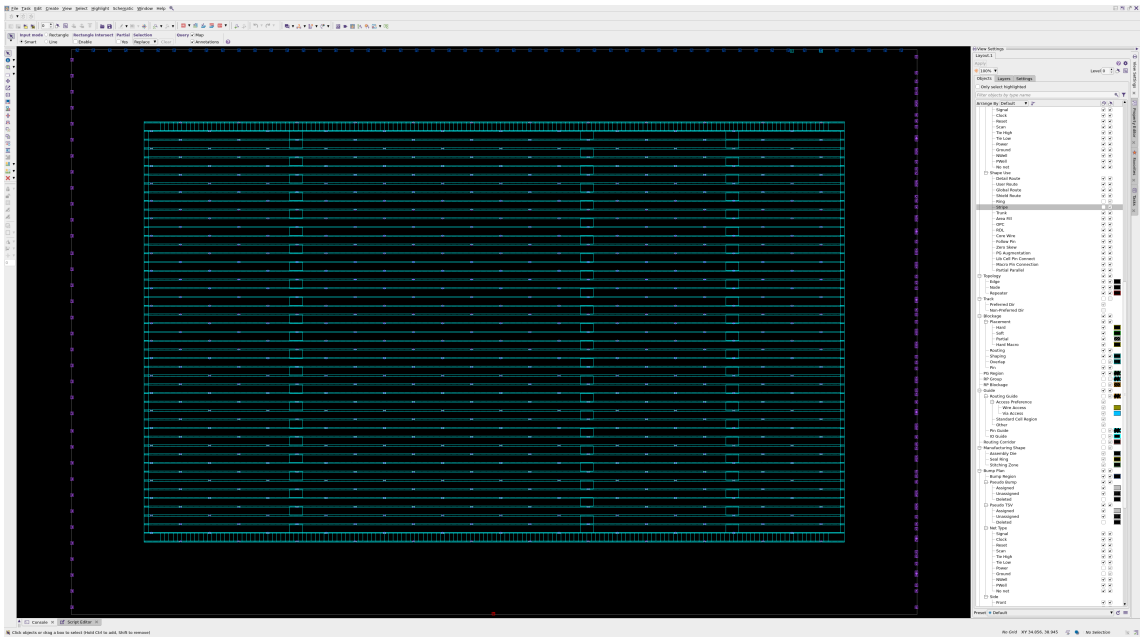

Figure 1.4: Final layout with power network. Power network is hidden, except for cell power.

Task 2. Cell placement

Procedure `analyzeDesign`:

```
13 proc analyzeDesign {DesignStage} {
14     variable ReportsDir
15 
16     redirect -file ${ReportsDir}/${DesignStage}_check_legality.rpt
    ↪ {check_legality}
17     redirect -file ${ReportsDir}/${DesignStage}_report_congestion.rpt
    ↪ {report_congestion}
18     redirect -file ${ReportsDir}/${DesignStage}_report_utilization.rpt
    ↪ {report_utilization}
19 }
```

Procedure used for making screenshots (Figures 2.1, 2.2 and 2.3):

```
21 proc make_layout_screenshot {suffix} {
22     variable TaskPrefix
23     variable LabDir
24     set ScreenshotsDir "${LabDir}/screenshots"
25 
26     gui_start
27     set top [gui_create_window -type TopLevel]
28     set layout [gui_create_window -type Layout -parent $top]
29     gui_show_window -window $top -show_state {maximized}
30     gui_show_window -window $layout -show_state {maximized}
31 
32     # Hide power network
33     gui_set_setting -window $layout -setting showRoutedPower -value false
34     gui_set_setting -window $layout -setting showRoutedGround -value false
35     # Hide layers with power networks (this includes pins)
36     gui_set_layout_layer_visibility {M6 M7} -window $layout -toggle
37 
38     gui_write_window_image -window $layout -clip -file
    ↪ ${ScreenshotsDir}/${TaskPrefix}floorplan_layout_${suffix}.png
39     gui_stop
40 }
```

Immediately after creating the placement blockage, `check_legality` reported 62 violations. All of them were caused by cells overlapping blockage. Later `legalize_placement -incremental` fixed all of these violations as can be seen in the Table 2.1 in column *with blockage*. `legalize_placement` warns about large displacements that could cause problems if the displaced cells are timing critical. The displacement is presented on helpful debug plots shown on Figures 2.4 and 2.5.

Analysis of data presented in reports (Table 2.1) shows that the *unified placement* entirely done by `compile_fusion` resulted in less overflow and smaller power usage (in the *Normal_Typical* scenario). Adding blockage and the subsequent legalisation caused the utilisation ratio to grow from approximately 66% (`core_utilization` was 0.65) to 69%, because part of the core area was excluded. Interestingly, despite

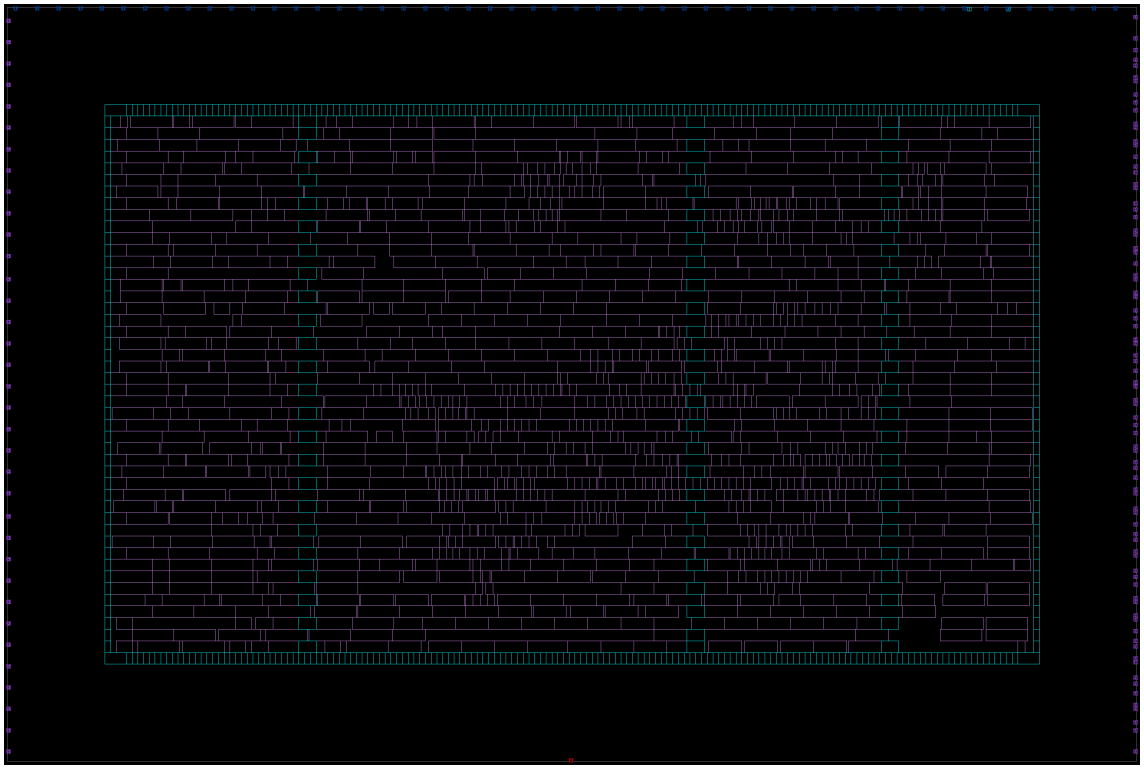

Figure 2.1: Layout (power network hidden) after `classic_placement` stage.

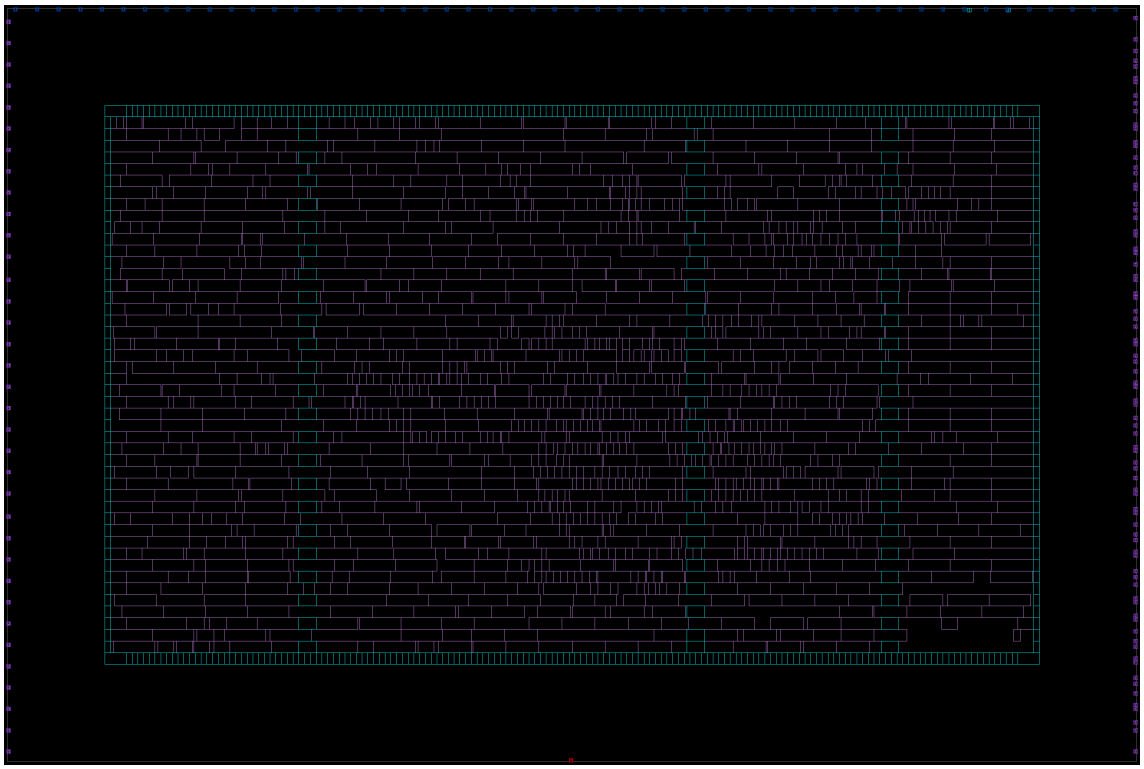

Figure 2.2: Layout (power network hidden) after `unified_placement` stage.

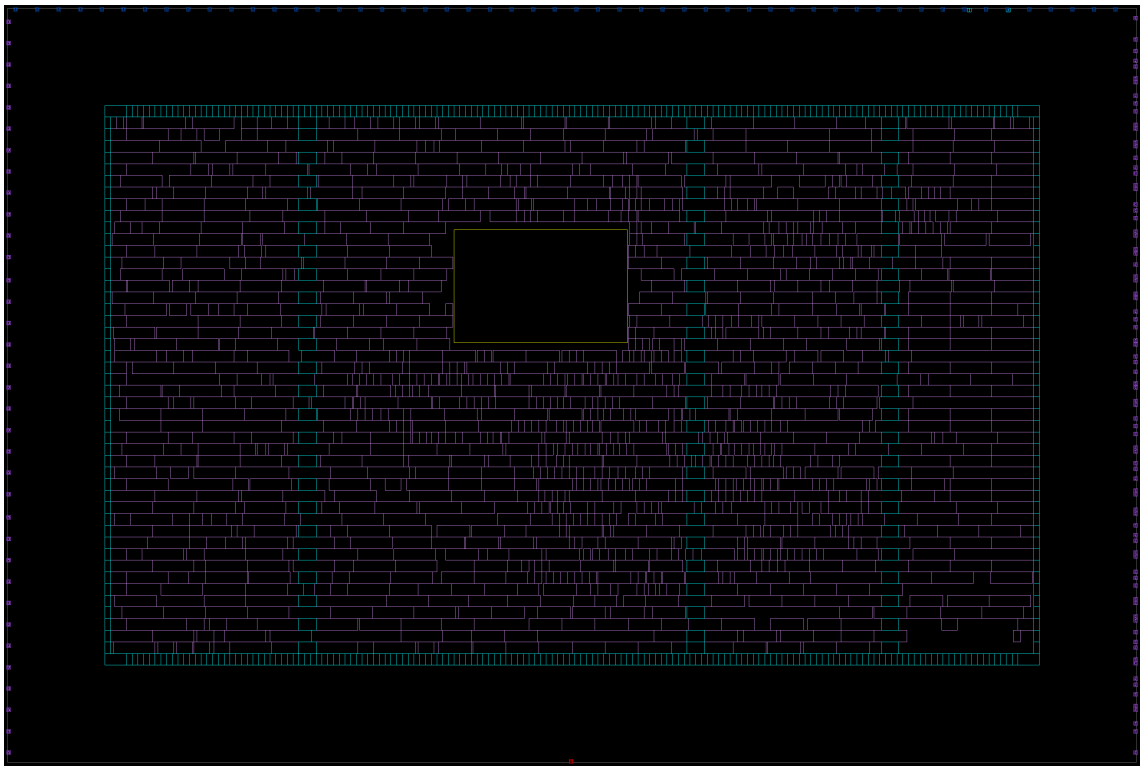

Figure 2.3: Layout (power network hidden) after `placement_with_blockage` stage.

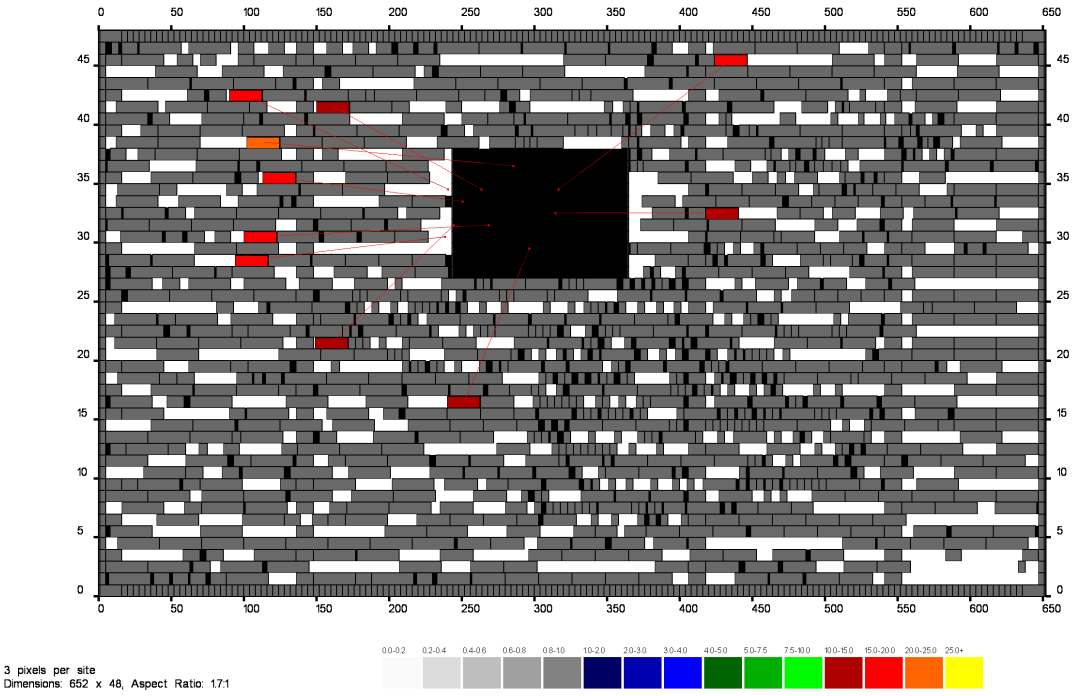

Figure 2.4: 10 cells with largest displacements during legalisation and their displacement vectors.

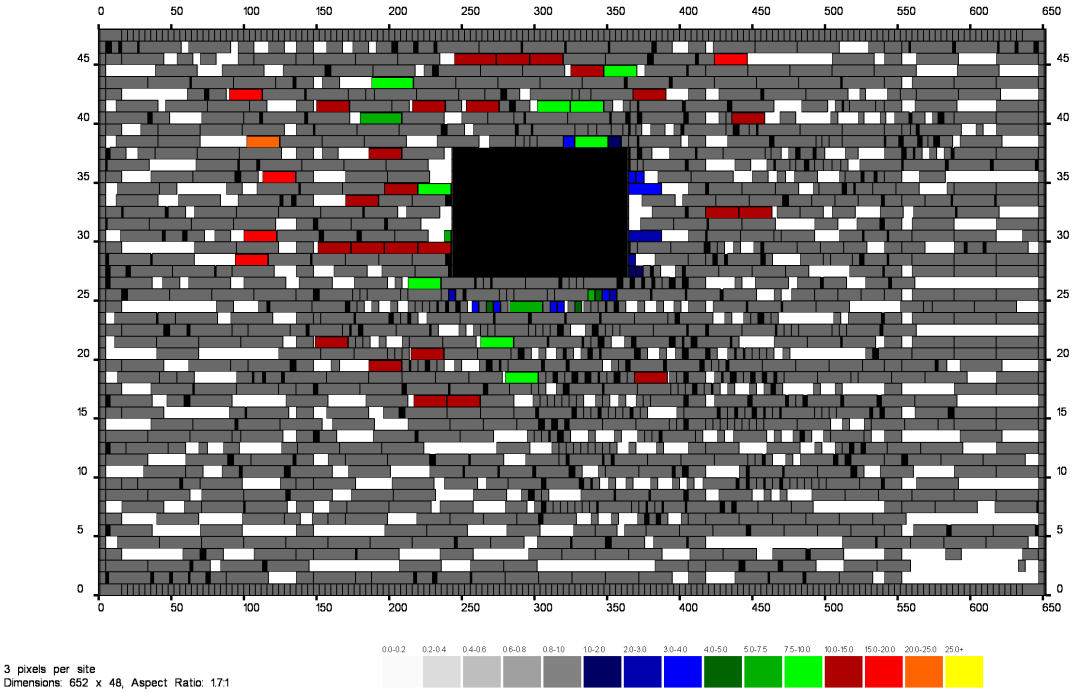

Figure 2.5: All cells displaced during legalisation, colour coded according to their displacement.

increasing the total net length by over 8%, timing and power remained the same. The power report contains messages informing that the activity for all scenarios was cached and no propagation is required. This seems to suggest that the data from power report has not been updated.

Later, when the library and `placement_and_optimization` block have been opened again, running `report_power` and `report_timing` produced different results. This means that the data from "with blockage" column in Table 2.1 is partially incorrect. Comparing the plotted max delay path from Fig. 2.6 with displaced cells from Figures 2.4 and 2.5 shows that the displacement caused by blockage and subsequent legalisation affected cells on the longest (timing-wise) path. The updated timing report, instead of 0 ns positive slack, shows setup violation by 0.08 ns. Similarly, power characteristics are worse, because the longer net causes net switching power to rise. These updated values are presented in the column marked *(rerun)*.

Table 2.1: Comparison of select parameters, depending on placement strategy. Presented power data for scenario *Normal_Typical*. Critical path length and worst setup slack are for scenario *Normal_Slow*. Hold slack is for the worst scenario (may differ). The *with blockage (rerun)* column uses reports generated after closing and reopening Fusion Compiler.

Parameter	classic	unified	with blockage	with blockage (rerun)
Legality violations	0	0	0	0
Total overflow	40	11	11	11
Overflowing GRCs %	0.22	0.07	0.07	0.07
Utilization ratio	0.6623	0.6622	0.6920	0.6920
Chip area	2254.280	2254.280	2254.280	2254.280
Core area	1385.2800	1385.2800	1385.2800	1385.2800
- Excluded area	0.0000	0.0000	59.5848	59.5848
- Total cell area	917.4372	917.3928	917.3928	917.3928
| - Combinational area	486.27	486.22	486.22	486.22
| - Noncombinational area	431.17	431.17	431.17	431.17
Power (pW)	7.12×10^7	7.08×10^7	7.08×10^7	7.16×10^7
- Dynamic power (pW)	7.09×10^7	7.05×10^7	7.05×10^7	7.13×10^7
| - Cell internal power (pW)	4.77×10^7	4.76×10^7	4.76×10^7	4.76×10^7
| - Net switching power (pW)	2.32×10^7	2.29×10^7	2.29×10^7	2.37×10^7
- Leakage power (pW)	3.17×10^5	3.16×10^5	3.16×10^5	3.16×10^5
Critical path length (ns)	1.73	1.78	1.78	1.87
Worst setup slack (ns)	0.00	0.00	0.00	−0.08
Worst hold slack (ns)	0.02	0.01	0.01	0.01
Net length	10 917.79	10 977.89	11 891.21	11 891.21
Number of nets	1647	1648	1648	1648
Number of cells	1215	1216	1216	1216
- Number of buffers	14	13	13	13
- Number of inverters	22	21	21	21

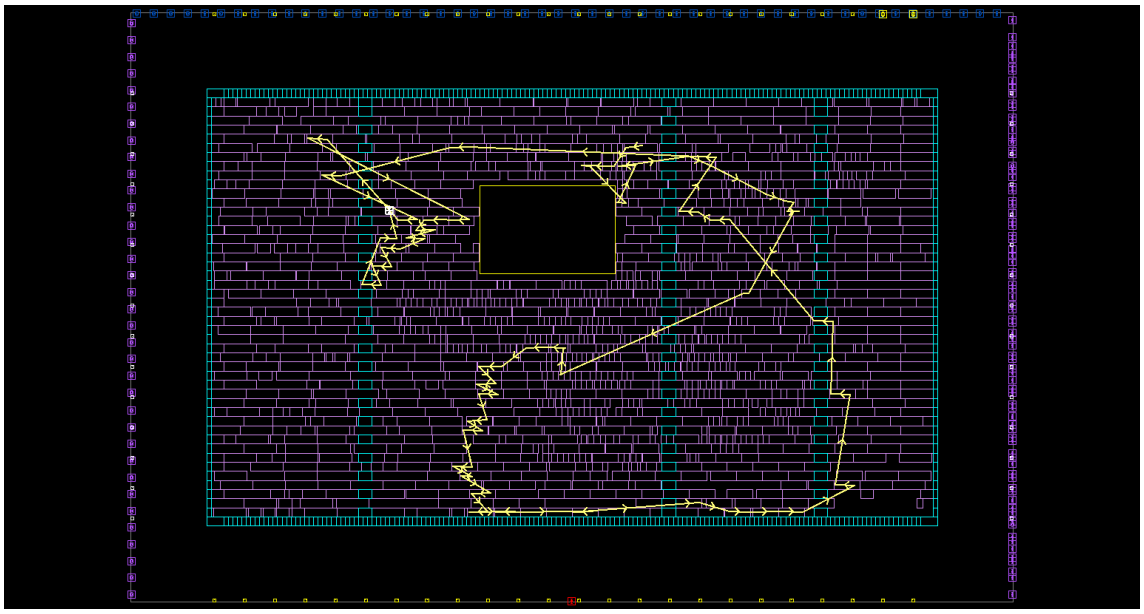

Figure 2.6: Max delay path of final `placement_and_optimization` block plotted on layout after reopening the design.

Task 3. Clock tree synthesis

Script `cts_cell_usage.tcl` was prepared according to the instruction:

```
#### Define cell usage during CTS

foreach stren {1 2 4 8 16 20} {
    set_lib_cell_purpose -include cts */SAEDRVT14_INV_${stren}
}
foreach stren {2 4 6 8 16 20} {
    set_lib_cell_purpose -include cts */SAEDRVT14_BUF_${stren}
}
```

As first noted by Mateusz during the laboratories, something seemed wrong and most of cells that were supposed to be used during CTS, could not be found in the layout. What `set_lib_cell_purpose -include cts` did was allowing those cells to be used for clock tree synthesis, but they were already allowed and so were many more cells. By default all cells have `included_purposes all`, which results in `valid_purposes {power hold cts optimization}`. This can be confirmed using attribute report (e.g. `report_attributes -nosplit -application [get_lib_cells */SAEDRVT14_BUF_16]`) or `get_attributes` with attribute names `valid_purposes`, `included_purposes` and `excluded_purposes` (e.g. `get_attribute -objects [get_lib_cells */SAEDRVT14_BUF_16] -name valid_purposes`). The actual list of cells considered for CTS (buffers, inverters and separately integrated clock gating cells) is printed by `clock_opt`:

```
Buffer/Inverter reference list for clock tree synthesis:
   saed14rvt_frame_timing/SAEDRVT14_BUF_10
   saed14rvt_frame_timing/SAEDRVT14_BUF_12
   saed14rvt_frame_timing/SAEDRVT14_BUF_16
   saed14rvt_frame_timing/SAEDRVT14_BUF_1P5
   saed14rvt_frame_timing/SAEDRVT14_BUF_1
   saed14rvt_frame_timing/SAEDRVT14_BUF_20
   saed14rvt_frame_timing/SAEDRVT14_BUF_2
   saed14rvt_frame_timing/SAEDRVT14_BUF_3
   saed14rvt_frame_timing/SAEDRVT14_BUF_4
   saed14rvt_frame_timing/SAEDRVT14_BUF_6
   saed14rvt_frame_timing/SAEDRVT14_BUF_8
   saed14rvt_frame_timing/SAEDRVT14_BUF_CDC_2
   saed14rvt_frame_timing/SAEDRVT14_BUF_CDC_4
   saed14rvt_frame_timing/SAEDRVT14_BUF_ECO_1
   saed14rvt_frame_timing/SAEDRVT14_BUF_ECO_2
   saed14rvt_frame_timing/SAEDRVT14_BUF_ECO_3
   saed14rvt_frame_timing/SAEDRVT14_BUF_ECO_4
   saed14rvt_frame_timing/SAEDRVT14_BUF_ECO_6
   saed14rvt_frame_timing/SAEDRVT14_BUF_ECO_7
   saed14rvt_frame_timing/SAEDRVT14_BUF_ECO_8
   saed14rvt_frame_timing/SAEDRVT14_BUF_S_0P5
   saed14rvt_frame_timing/SAEDRVT14_BUF_S_0P75
   saed14rvt_frame_timing/SAEDRVT14_BUF_S_10
```

saed14rvt_frame_timing/SAEDRVT14_BUF_S_12
saed14rvt_frame_timing/SAEDRVT14_BUF_S_16
saed14rvt_frame_timing/SAEDRVT14_BUF_S_1P5
saed14rvt_frame_timing/SAEDRVT14_BUF_S_1
saed14rvt_frame_timing/SAEDRVT14_BUF_S_20
saed14rvt_frame_timing/SAEDRVT14_BUF_S_2
saed14rvt_frame_timing/SAEDRVT14_BUF_S_3
saed14rvt_frame_timing/SAEDRVT14_BUF_S_4
saed14rvt_frame_timing/SAEDRVT14_BUF_S_6
saed14rvt_frame_timing/SAEDRVT14_BUF_S_8
saed14rvt_frame_timing/SAEDRVT14_BUF_U_0P5
saed14rvt_frame_timing/SAEDRVT14_BUF_U_0P75
saed14rvt_frame_timing/SAEDRVT14_BUF_UCDC_0P5
saed14rvt_frame_timing/SAEDRVT14_BUF_UCDC_1
saed14rvt_frame_timing/SAEDRVT14_DEL_L4D100_1
saed14rvt_frame_timing/SAEDRVT14_DEL_L4D100_2
saed14rvt_frame_timing/SAEDRVT14_DEL_R2V1_1
saed14rvt_frame_timing/SAEDRVT14_DEL_R2V1_2
saed14rvt_frame_timing/SAEDRVT14_DEL_R2V2_1
saed14rvt_frame_timing/SAEDRVT14_DEL_R2V2_2
saed14rvt_frame_timing/SAEDRVT14_DEL_R2V3_1
saed14rvt_frame_timing/SAEDRVT14_DEL_R2V3_2
saed14rvt_frame_timing/SAEDRVT14_BUF_PECO_12
saed14rvt_frame_timing/SAEDRVT14_BUF_PECO_1
saed14rvt_frame_timing/SAEDRVT14_BUF_PECO_2
saed14rvt_frame_timing/SAEDRVT14_BUF_PECO_4
saed14rvt_frame_timing/SAEDRVT14_BUF_PECO_8
saed14rvt_frame_timing/SAEDRVT14_BUF_PS_0P75
saed14rvt_frame_timing/SAEDRVT14_BUF_PS_1P5
saed14rvt_frame_timing/SAEDRVT14_BUF_PS_3
saed14rvt_frame_timing/SAEDRVT14_BUF_PS_6
saed14rvt_frame_timing/SAEDRVT14_DEL_PR2V2_1
saed14rvt_frame_timing/SAEDRVT14_AOBUF_IW_0P75
saed14rvt_frame_timing/SAEDRVT14_AOBUF_IW_1P5
saed14rvt_frame_timing/SAEDRVT14_AOBUF_IW_3
saed14rvt_frame_timing/SAEDRVT14_AOBUF_IW_6
saed14rvt_frame_timing/SAEDRVT14_INV_0P5
saed14rvt_frame_timing/SAEDRVT14_INV_0P75
saed14rvt_frame_timing/SAEDRVT14_INV_10
saed14rvt_frame_timing/SAEDRVT14_INV_12
saed14rvt_frame_timing/SAEDRVT14_INV_16
saed14rvt_frame_timing/SAEDRVT14_INV_1P5
saed14rvt_frame_timing/SAEDRVT14_INV_1
saed14rvt_frame_timing/SAEDRVT14_INV_20
saed14rvt_frame_timing/SAEDRVT14_INV_2
saed14rvt_frame_timing/SAEDRVT14_INV_3
saed14rvt_frame_timing/SAEDRVT14_INV_4
saed14rvt_frame_timing/SAEDRVT14_INV_6
saed14rvt_frame_timing/SAEDRVT14_INV_8
saed14rvt_frame_timing/SAEDRVT14_INV_ECO_1
saed14rvt_frame_timing/SAEDRVT14_INV_ECO_2
saed14rvt_frame_timing/SAEDRVT14_INV_ECO_3
saed14rvt_frame_timing/SAEDRVT14_INV_ECO_4
saed14rvt_frame_timing/SAEDRVT14_INV_ECO_6
saed14rvt_frame_timing/SAEDRVT14_INV_ECO_8
saed14rvt_frame_timing/SAEDRVT14_INV_S_0P5
saed14rvt_frame_timing/SAEDRVT14_INV_S_0P75

```
   saed14rvt_frame_timing/SAEDRVT14_INV_S_10
   saed14rvt_frame_timing/SAEDRVT14_INV_S_12
   saed14rvt_frame_timing/SAEDRVT14_INV_S_16
   saed14rvt_frame_timing/SAEDRVT14_INV_S_1P5
   saed14rvt_frame_timing/SAEDRVT14_INV_S_1
   saed14rvt_frame_timing/SAEDRVT14_INV_S_20
   saed14rvt_frame_timing/SAEDRVT14_INV_S_2
   saed14rvt_frame_timing/SAEDRVT14_INV_S_3
   saed14rvt_frame_timing/SAEDRVT14_INV_S_4
   saed14rvt_frame_timing/SAEDRVT14_INV_S_5
   saed14rvt_frame_timing/SAEDRVT14_INV_S_6
   saed14rvt_frame_timing/SAEDRVT14_INV_S_7
   saed14rvt_frame_timing/SAEDRVT14_INV_S_8
   saed14rvt_frame_timing/SAEDRVT14_INV_S_9
   saed14rvt_frame_timing/SAEDRVT14_INV_PECO_12
   saed14rvt_frame_timing/SAEDRVT14_INV_PECO_1
   saed14rvt_frame_timing/SAEDRVT14_INV_PECO_2
   saed14rvt_frame_timing/SAEDRVT14_INV_PECO_4
   saed14rvt_frame_timing/SAEDRVT14_INV_PECO_8
   saed14rvt_frame_timing/SAEDRVT14_INV_PS_1
   saed14rvt_frame_timing/SAEDRVT14_INV_PS_2
   saed14rvt_frame_timing/SAEDRVT14_INV_PS_3
   saed14rvt_frame_timing/SAEDRVT14_INV_PS_6

ICG reference list:
   saed14rvt_frame_timing/SAEDRVT14_CKGTNLT_V5_12
   saed14rvt_frame_timing/SAEDRVT14_CKGTNLT_V5_1
   saed14rvt_frame_timing/SAEDRVT14_CKGTNLT_V5_2
   saed14rvt_frame_timing/SAEDRVT14_CKGTNLT_V5_3
   saed14rvt_frame_timing/SAEDRVT14_CKGTNLT_V5_4
   saed14rvt_frame_timing/SAEDRVT14_CKGTNLT_V5_5
   saed14rvt_frame_timing/SAEDRVT14_CKGTNLT_V5_6
   saed14rvt_frame_timing/SAEDRVT14_CKGTNLT_V5_8
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_12
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_16
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_1
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_20
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_24
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_2
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_3
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_4
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_5
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_6
   saed14rvt_frame_timing/SAEDRVT14_CKGTPLT_V5_8
   saed14rvt_frame_timing/SAEDRVT14_CKGTPL_V5_0P5
   saed14rvt_frame_timing/SAEDRVT14_CKGTPL_V5_1
   saed14rvt_frame_timing/SAEDRVT14_CKGTPL_V5_2
   saed14rvt_frame_timing/SAEDRVT14_CKGTPL_V5_4
   saed14rvt_frame_timing/SAEDRVT14_CKINVGTPLT_V7_1
   saed14rvt_frame_timing/SAEDRVT14_CKINVGTPLT_V7_2
   saed14rvt_frame_timing/SAEDRVT14_CKINVGTPLT_V7_3
   saed14rvt_frame_timing/SAEDRVT14_CKINVGTPLT_V7_4
   saed14rvt_frame_timing/SAEDRVT14_CKINVGTPLT_V7_5
   saed14rvt_frame_timing/SAEDRVT14_CKINVGTPLT_V7_6
   saed14rvt_frame_timing/SAEDRVT14_CKINVGTPLT_V7_8
```

The `set_driving_cell` command used to set attributes of the `clk` port is also used

incorrectly. Documentation of this command says:

> Driving cell data is managed on a **per-scenario basis**. For designs with multiple scenarios, you can specify different driving cell settings for different scenarios by using -modes, -corners, and -scenarios options.
>
> By default, if none of these options are specified, **the command applies to the current scenario**. The tool issues an error if there is no current scenario. If the -scenarios option is given, the command applies to all of the specified scenarios.

This can be confirmed using `report_ports -drive clk` and changing scenarios:

```
fc_shell> report_ports -drive clk
****************************************
Report : port
        -drive
Module : dut_toplevel
Mode   : PowerSave
Corner : Slow
Scenario: PowerSave_Slow
Version: V-2023.12
Date   : Tue Jul  1 14:42:21 2025
****************************************

            Resistance (min/max)
Input Port  Rise             Fall
--------------------------------------------------------------------------------
clk         --               --

                     Driving Cell
Input Port     Type     Cell                Mult      Clock     Attrs
--------------------------------------------------------------------------------
clk            min_rise SAEDRVT14_INV_20
clk            min_fall SAEDRVT14_INV_20
clk            max_rise SAEDRVT14_INV_20
clk            max_fall SAEDRVT14_INV_20
1
fc_shell> current_scenario Normal_Typical
{Normal_Typical}
fc_shell> report_ports -drive clk
****************************************
Report : port
        -drive
Module : dut_toplevel
Mode   : Normal
Corner : Typical
Scenario: Normal_Typical
Version: V-2023.12
Date   : Tue Jul  1 14:42:39 2025
****************************************

            Resistance (min/max)
Input Port  Rise             Fall
--------------------------------------------------------------------------------
clk         --               --
1
```

This might cause problems with parameter analysis in scenarios other than *PowerSave_Slow*, but I did not try synthesis with `set_driving_cell -lib_cell ${CellPrefix}_INV_20 [get_ports ${ClockName}] -scenarios [all_scenarios]` to see if there are any differences.

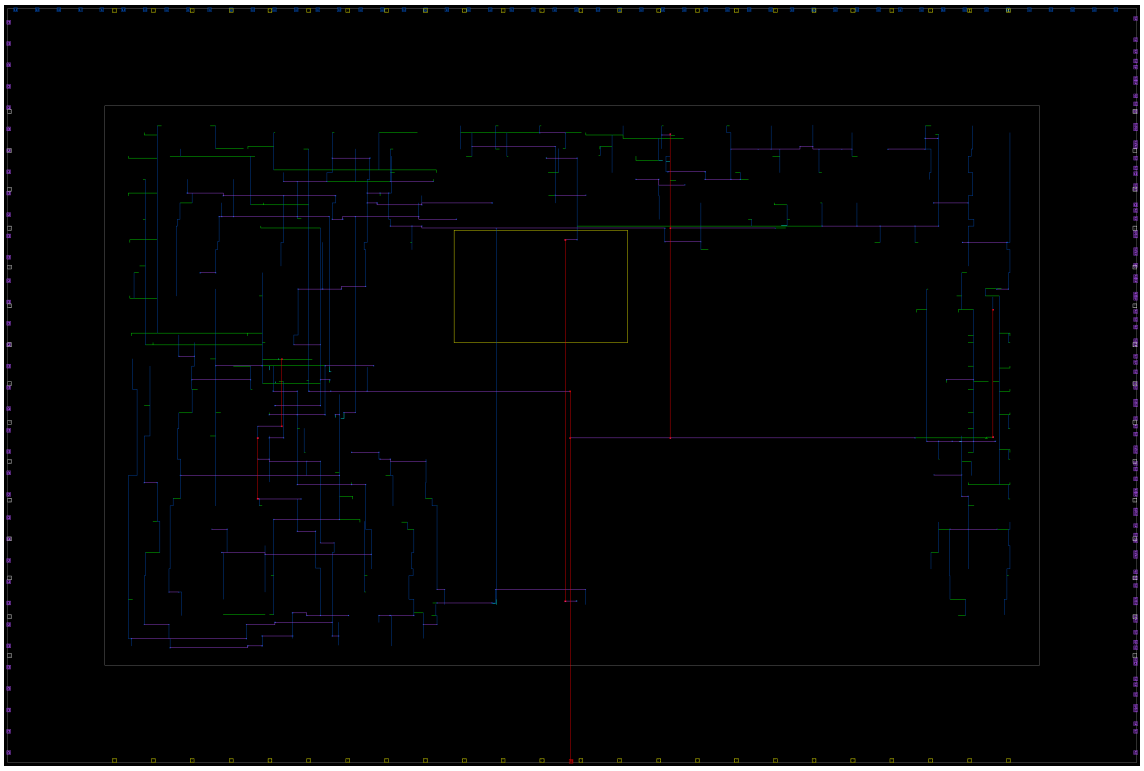

Figure 3.1: Clock routing after `build_clock` phase.

Fig. 3.6 shows that the ground shield around clock exists mostly on M5 (vertical), but also partially in small fragments on one side of the clock on other layers (M6, M4, M3, M2, M1). Other figures show how clock is gradually being routed - starting from straight lines to finally snap to tracks of different widths.

Table 3.1: Circuit parameters after clock tree synthesis. Generated in the same way as Table 2.1.

Parameter	clock tree synthesis
Legality violations	0
Total overflow	1
Overflowing GRCs %	0.01
Utilization ratio	0.7014
Chip area	2254.280
Core area	1385.2800
- Excluded area	59.5848
- Total cell area	929.8248
| - Combinational area	498.39

Parameter	clock tree synthesis
| - Noncombinational area	431.43
Power (Normal_Typical) (pW)	6.95×10^7
- Dynamic power (pW)	6.92×10^7
| - Cell internal power (pW)	3.95×10^7
| - Net switching power (pW)	2.97×10^7
- Leakage power (pW)	3.22×10^5
Critical path length (ns)	1.67
Worst setup slack (ns)	0.05
Worst hold slack (ns)	−0.00
Net length	12 218.91
Number of nets	1675
Number of cells	1251
- Number of buffers	36
- Number of inverters	23

Analysis of parameters from Table 3.1 shows greately increased net length and number of buffers caused by adding the clock tree. Other changes from Table 2.1 include lower power, caused by lower cell internal power, although the net switching power increased. Timing has also improved significantly, but now instead of setup violation there is a small hold violation (rounded to 0.00 ns).

3.1 Task 3 script

The script was modified to automatically make screenshots with different settings:

```
1  ##########################################################################
2  ###   Task 3 - Clock Tree Synthesis
3  ##########################################################################
4
5  set TaskPrefix "task3_"
6  set DesignStage clock_tree_synthesis
7
8  #### Sourcing main setup script
9  source -echo  ../../../setup/main_setup.tcl
10
11 #### Sourcing project specific setup script
12 source -echo  ${SetupDir}/design_setup.tcl
13
14 set ScreenshotsDir "${LabDir}/screenshots"
15
16 # Reuse top level window
17 gui_start
18 set top [gui_create_window -type TopLevel -show_state maximized]
19
20 # https://docs.amd.com/r/en-US/ug894-vivado-tcl-scripting/Parsing-Command-Line-
   ↪ Arguments
21 proc lshift  {listVar} {
22     upvar 1 $listVar L
```

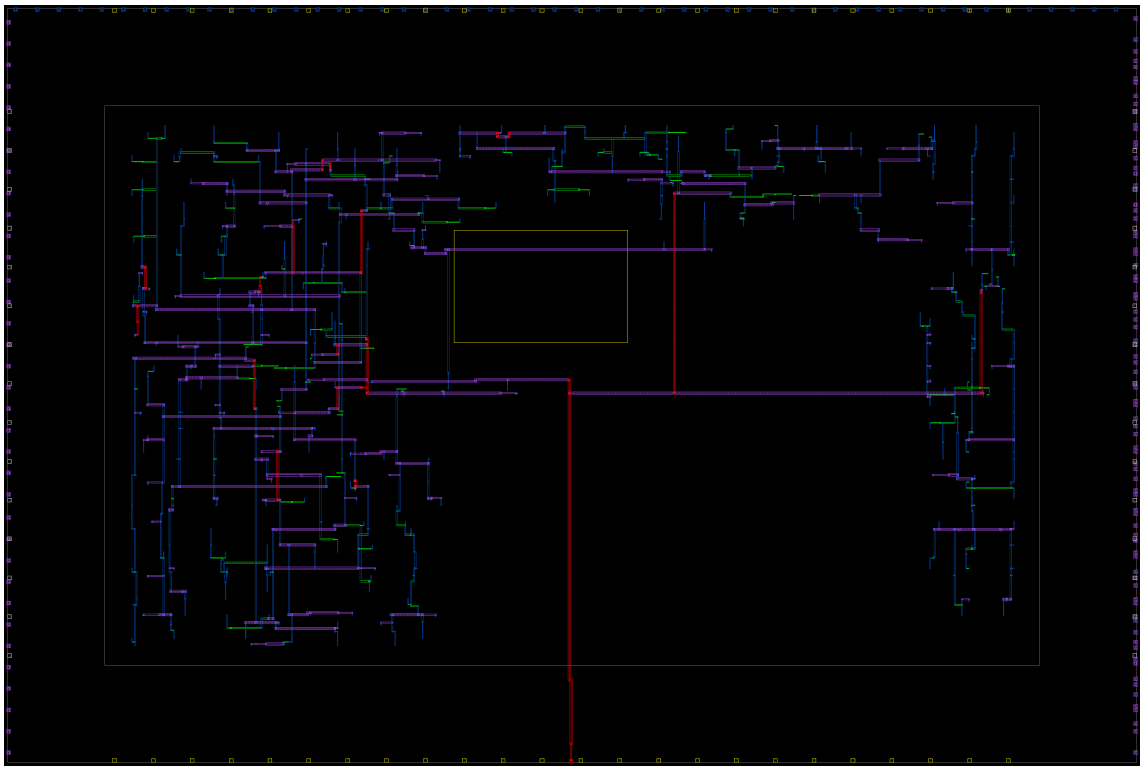

Figure 3.2: Clock routing after `route_clock` phase.

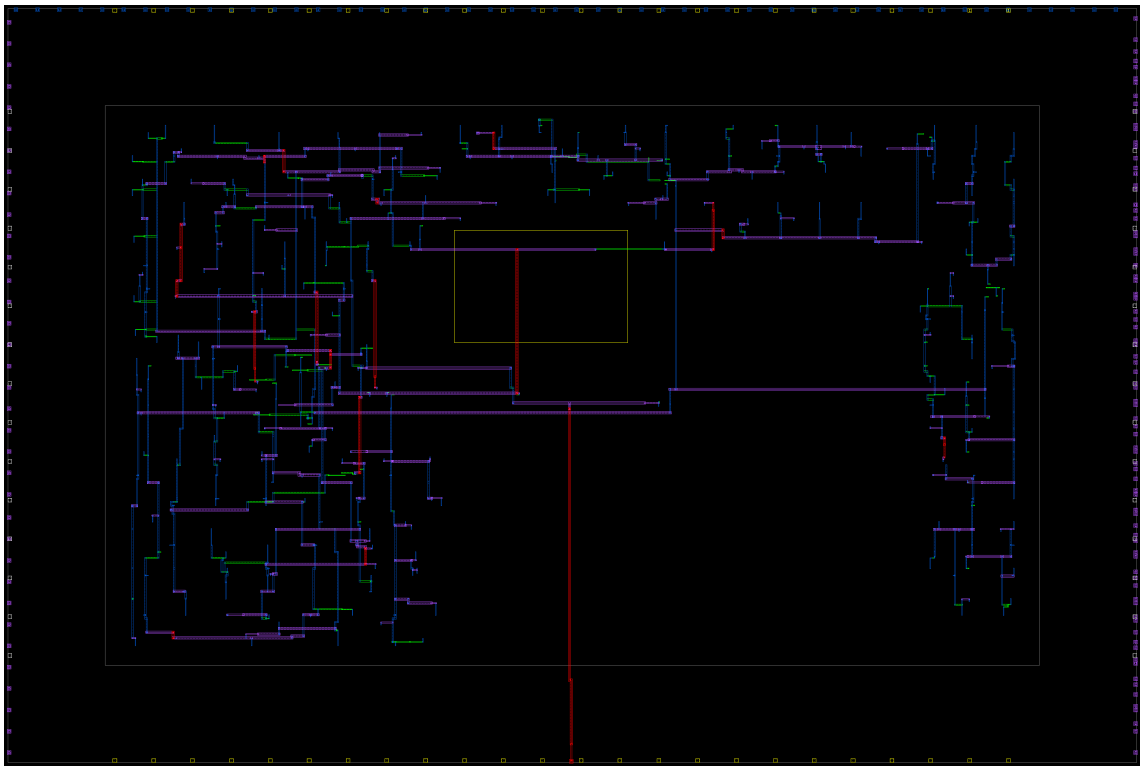

Figure 3.3: Clock routing after `final_opto` phase, without global routes.

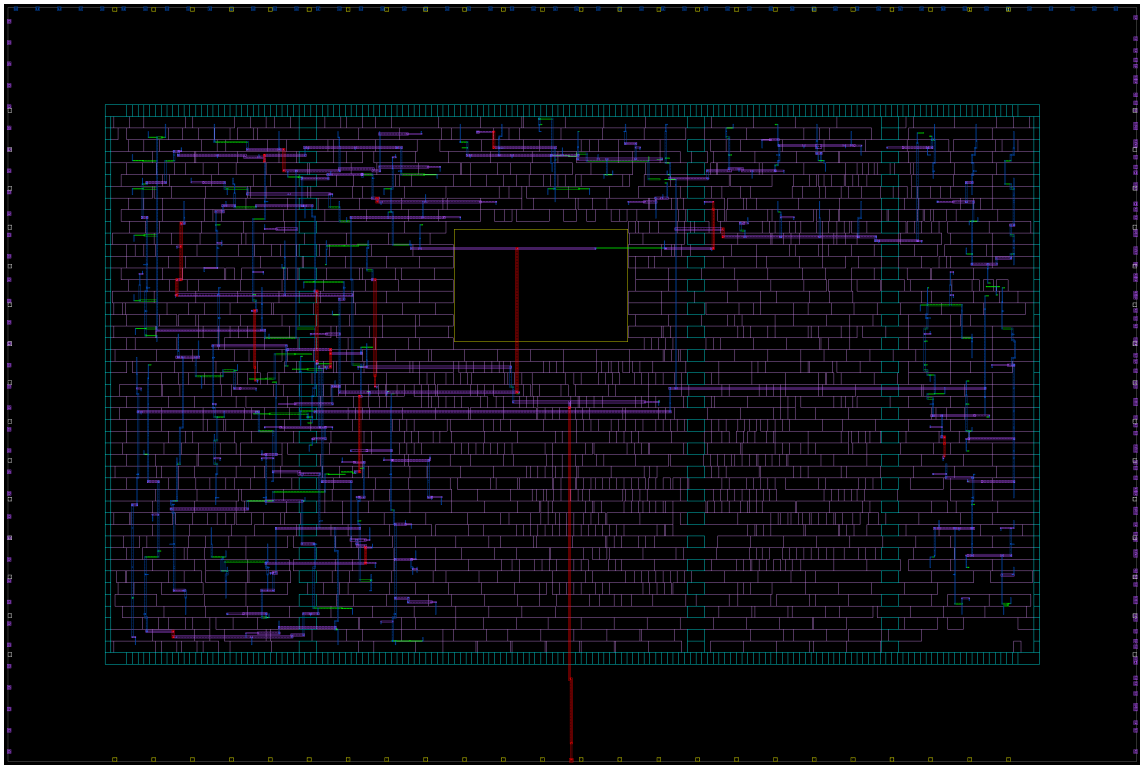

Figure 3.4: Clock routing after `final_opto` phase, with standard cells visible and global routes hidden.

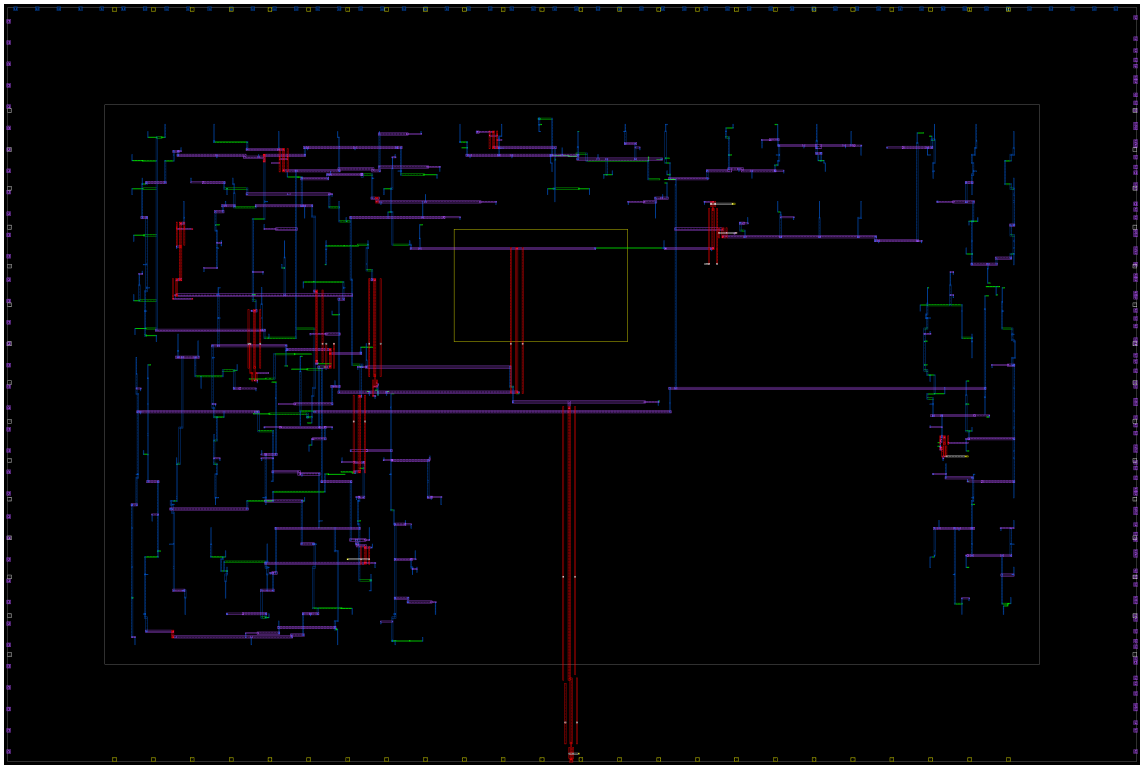

Figure 3.5: Clock routing and ground shields. Rings, straps, standard cells and their power are hidden.

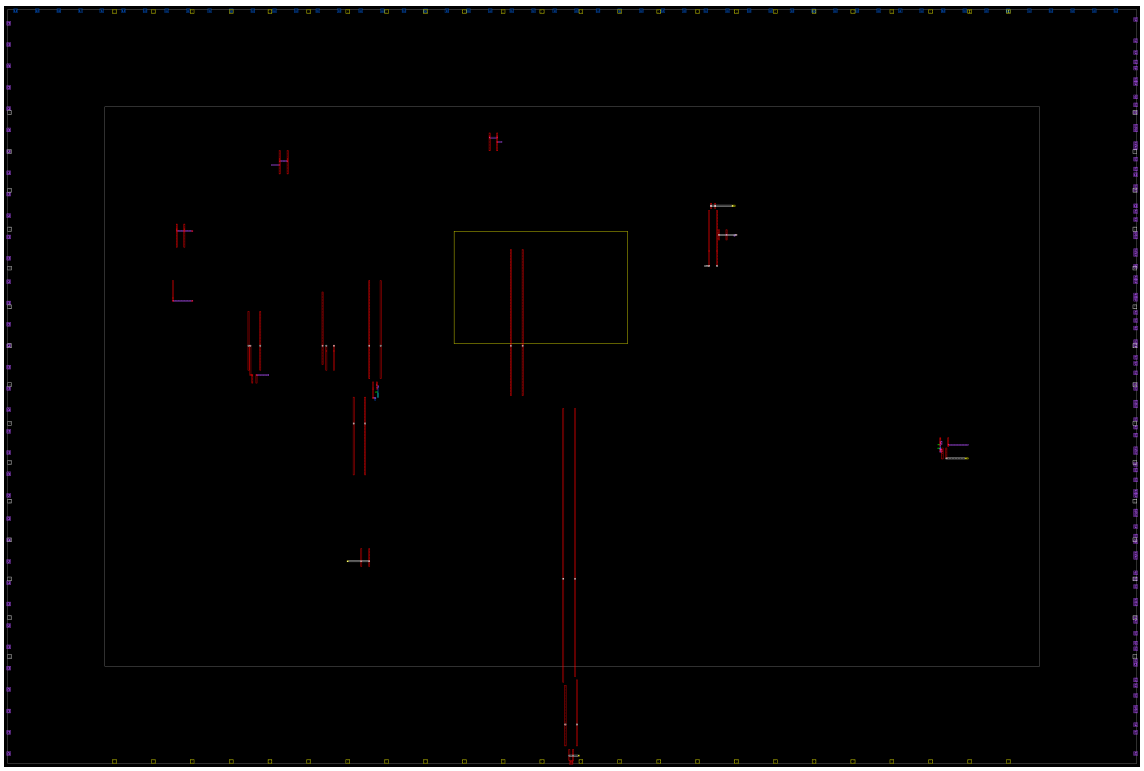

Figure 3.6: Ground shields only, with clock hidden.

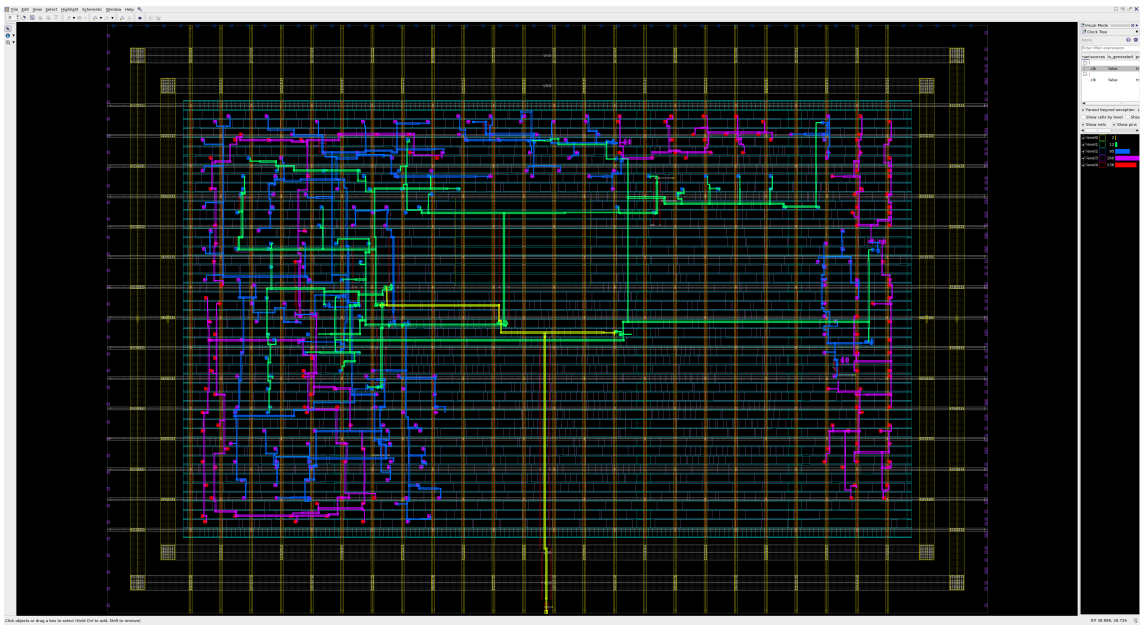

Figure 3.7: Highlighted clocks tree at the end of the task.

```
23      set r [lindex $L 0]
24      set L [lreplace $L [set L 0] 0]
25      return $r
26  }
27
28  proc make_layout_screenshot {suffix args} {
29      variable TaskPrefix
30      variable ScreenshotsDir
31      variable top
32      set hidePower 1
33      set hideCells 0
34      set hideNonShieldPower 0
35      set hideClock 0
36      set hideGlobalRouting 0
37
38      while {[llength $args]} {
39          set flag [lshift args]
40          switch -exact -- $flag {
41              -showPower {
42                  set hidePower 0
43              }
44              -hideNonShieldPower {
45                  set hideNonShieldPower 1
46              }
47              -hideCells {
48                  set hideCells 1
49              }
50              -hideClock {
51                  set hideClock 1
52              }
53              -hideGlobalRouting {
54                  set hideGlobalRouting 1
55              }
56          }
57      }
58
59      if {$hidePower && $hideNonShieldPower} {
60          puts "-hideNonShieldPower without -showPower will hide shields too,
             ↪ because -showPower has higher priority"
61      }
62
63      # gui_start
64      # set top [gui_create_window -type TopLevel]
65      set layout [gui_create_window -type Layout -parent $top]
66      gui_show_window -window $top -show_state {maximized}
67      gui_show_window -window $layout -show_state {maximized}
68
69      if {$hidePower} {
70          # Hide power network
71          gui_set_setting -window $layout -setting showRoutedPower -value false
72          gui_set_setting -window $layout -setting showRoutedGround -value false
73      }
74      if {$hideNonShieldPower} {
75          # Hide power and ground ring, straps and cell connections, but not
             ↪ shields
76          # Should be used with showPower
77          gui_set_setting -window $layout -setting showRoutedRing -value false
```

```
78          gui_set_setting -window $layout -setting showRoutedStrap -value false
79          gui_set_setting -window $layout -setting showRoutedPinConStd -value false
80      }
81          if {$hideCells} {
82                  # Hide cells for clearer screenshots
83                  gui_set_setting -window $layout -setting showCell -value false
84          }
85      if {$hideClock} {
86          # Hide clock (to see only shields)
87          gui_set_setting -window $layout -setting showRoutedClock -value false
88      }
89      if {$hideGlobalRouting} {
90          gui_set_setting -window $layout -setting showRoutedGRoute -value false
91      }
92
93      gui_write_window_image -window $layout -clip -file
    ↪ ${ScreenshotsDir}/${TaskPrefix}floorplan_layout_${suffix}.png
94      gui_close_window -window $layout
95      # gui_stop
96  }
97
98  # Open the design library
99  open_lib ${ResultsDir}/${DesignLibrary}
100
101 # Copy and open block
102 copy_block -from ${DesignName}/placement_and_optimization -to
    ↪ ${DesignName}/clock_tree_synthesis
103 open_block ${DesignName}/clock_tree_synthesis
104
105 # Setup application options
106 source -echo ../scripts/set_cts_options.tcl
107
108 #### Specify the driving cell
109 set_driving_cell -lib_cell ${CellPrefix}_INV_20 [get_ports ${ClockName}]
110
111 #### Define cell usage during CTS
112 source -echo ../scripts/cts_cell_usage.tcl
113
114 #### Create Shielding options and Non-Default Routing (NDR) rules
115
116 source -echo ../scripts/clock_ndr.tcl
117
118 #### Set target skew value
119 set_clock_tree_options -clocks [all_clocks] \
120         -target_skew 0.05
121
122 #### clock_opt flow
123 get_clocks
124
125 # List the stages of clock_opt command
126 clock_opt -list_only
127
128 # Synthesize and optimize the clock tree
129 clock_opt -to build_clock
130 make_layout_screenshot build_clock -hideCells
131
132 # Detail routing of clock
```

```
133 clock_opt -from build_clock -to route_clock
134 make_layout_screenshot route_clock -hideCells
135
136 # Optimization and legalization
137 clock_opt -to final_opto
138 make_layout_screenshot final_opto_with_global_routing -hideCells
139 make_layout_screenshot final_opto -hideCells -hideGlobalRouting
140 make_layout_screenshot final_opto_wcells -hideGlobalRouting
141
142 # Remove global routes to review the clock tree
143 remove_routes -global_route
144 make_layout_screenshot final_opto_gr_removed -hideCells
145
146 #### Clock shielding with VSS
147 set clock_nets [get_nets -hierarchical -filter "net_type == clock"]
148 create_shields -nets ${clock_nets} -with_ground VSS -preferred_direction_only
    ↪ true -align_to_shape_end true
149
150 #### Connect PG nets
151 connect_pg_net -net VDD [get_pins -hierarchical  */VDD]
152 connect_pg_net -net VSS [get_pins -hierarchical  */VSS]
153
154 # Analyze the design
155 check_legality
156 report_congestion
157 report_utilization
158 redirect -file ${ReportsDir}/${DesignStage}_check_legality.rpt {check_legality}
159 redirect -file ${ReportsDir}/${DesignStage}_report_congestion.rpt
    ↪ {report_congestion}
160 redirect -file ${ReportsDir}/${DesignStage}_report_utilization.rpt
    ↪ {report_utilization}
161 generateReports ${DesignStage}
162
163 make_layout_screenshot shielded -hideCells -showPower -hideNonShieldPower
164 make_layout_screenshot shielded_wcells -showPower -hideNonShieldPower
165 make_layout_screenshot shield_only -hideCells -showPower -hideNonShieldPower
    ↪ -hideClock
166
167 # ~~Highlight by clock tree manually and make a screenshot~~
168 # I love undocumented options that don't appear in autocomplete lists, man page
    ↪ viewer or both...
169 # Grepping doc/ FTW
170 # gui_start
171 # set top [gui_create_window -type TopLevel -show_state maximized]
172 set layout [gui_create_window -type Layout -parent $top -show_state maximized]
173 # Well, it looks like it being missing in multiple places meant that it does not
    ↪ really exist (except for `man gui_set_layout_visual_mode` which works...)
174 # gui_set_layout_visual_mode -window $layout -mode "Clock Tree"
175 gui_show_map -map "Clock Tree" -show true -window $layout
176 gui_set_map_option -map "Clock Tree" -option clock -value "Normal:clk"
177 gui_write_window_image -window $layout -clip -file
    ↪ ${ScreenshotsDir}/${TaskPrefix}clock_tree_vm.png
178 gui_stop
179
180 get_blocks -all
181 list_blocks
182
```

```
183 save_block
184 save_lib
185
186 close_blocks
187 close_lib
188
189 exit
```

Task 4. Routing

Instruction says that the script contains an example of command creating routing blockage on layers *M2*, *M3* and *M4* that has "?" signs, which should be replaced with placement blockage coordinates from task 2. However, the script contained a command with already filled proper coordinates and only layer *M4*. There was also a comment with "?" signs instead of coordinates and it only had *M4* as the layer affected by blockage. Since the instruction did not ask for more changes, the commands were not modified:

```
58 #### Routing blockage example
59 create_routing_blockage -boundary {{22.9780 21.6000} {22.9780 27.4000} {31.8940
    ↪  27.4000} {31.8940 21.6000}} -net_types {signal} -layers {M4} -name_prefix RB
    ↪  -zero_spacing
60 # create_routing_blockage -boundary {{? ?} {? ?} {? ?} {? ?}} -net_types {signal}
    ↪  -layers {M4} -name_prefix RB -zero_spacing
```

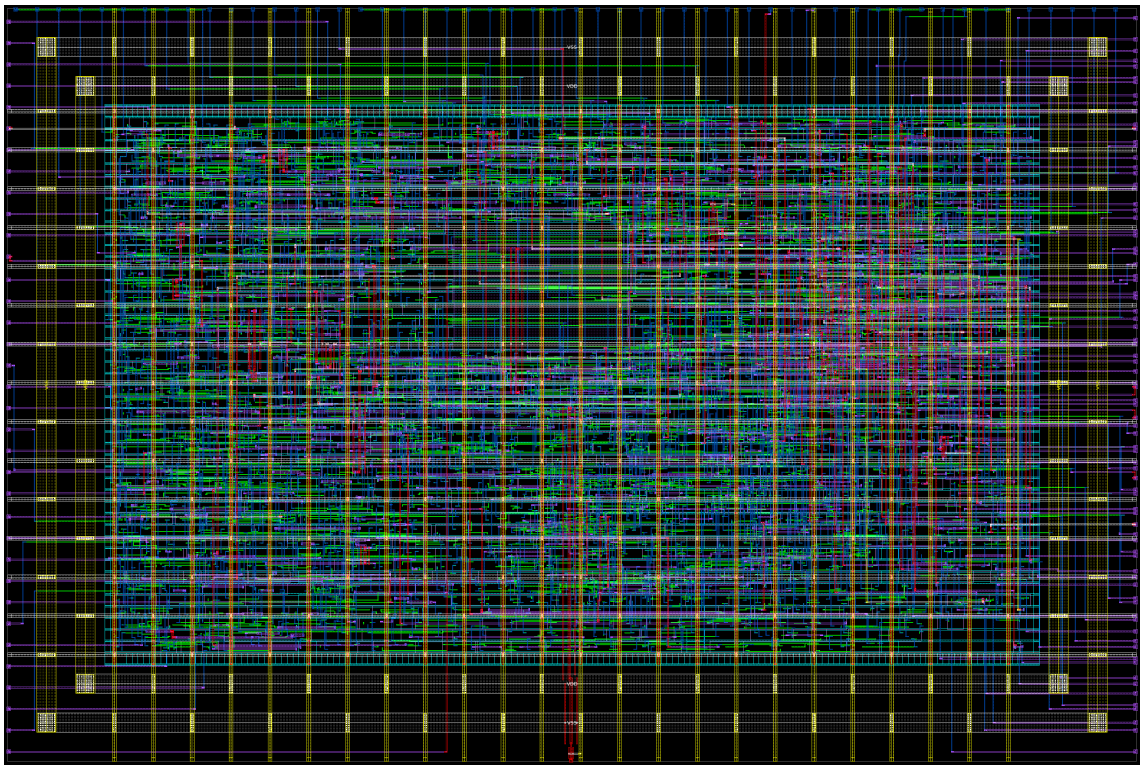

Figure 4.1: Layout view after routing.

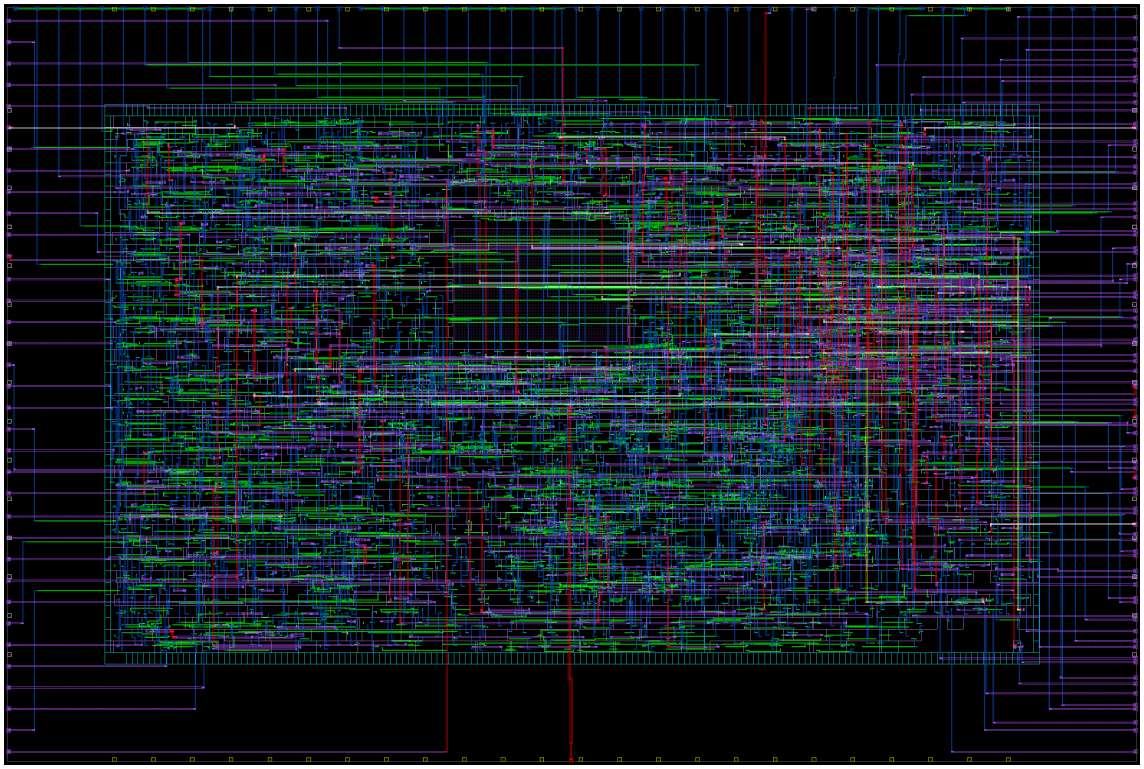

Figure 4.2: Layout view after routing with power and ground routes hidden.

Table 4.1: Circuit parameters after clock tree synthesis.
Generated in the same way as Tables 2.1 and 3.1.

Parameter	routing with blockage
Legality violations	0
Total overflow	58
Overflowing GRCs %	0.39
Utilization ratio	0.7015
Chip area	2254.280
Core area	1385.2800
- Excluded area	59.5848
- Total cell area	930.0024
| - Combinational area	498.57
| - Noncombinational area	431.43
Power (pW)	6.88×10^7
- Dynamic power (pW)	6.85×10^7
| - Cell internal power (pW)	3.95×10^7
| - Net switching power (pW)	2.90×10^7
- Leakage power (pW)	3.21×10^5
Critical path length (ns)	1.73
Worst setup slack (ns)	−0.02
Worst hold slack (ns)	−0.00
Net length	11 799.47

Parameter	routing with blockage
Number of nets	1676
Number of cells	1252
- Number of buffers	37
- Number of inverters	23

Interesting details observed in logs and reports:

- `sizeof_collection [get_nets -hierarchical *]` from before routing reported 1675 routes, but there is 1 net more in area report after routing.
- There are 2523 off track pins.
- ECO did not change anything, because there were no violations.
- Now both setup and hold times are violated.

Task 5. Signoff

Only differences that can be observed between parameters from this (Table 5.1) and the previous step (Table 4.1) are:

- net length increased by 3.29 microns,
- no area is excluded (`remove_placement_blockages -all`),
- utilization ratio decreased, because there is no excluded area.

The final circuit does not meet our timing requirements—both setup and hold times are violated.

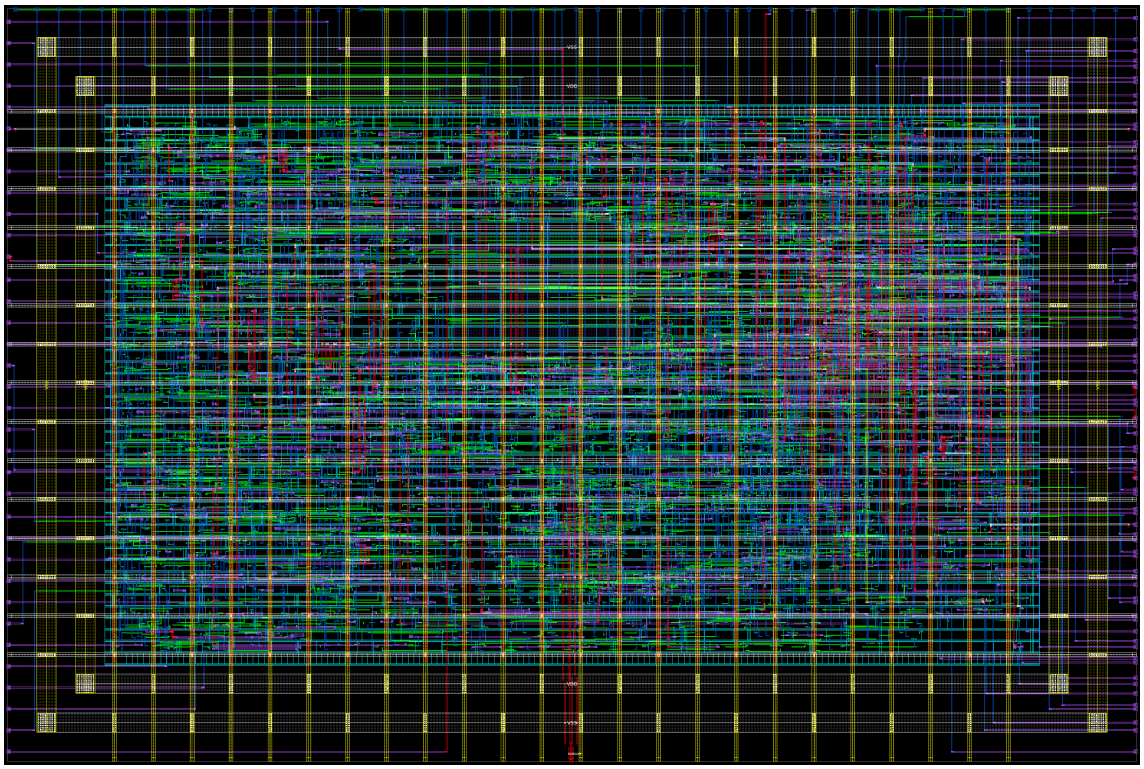

Figure 5.1: Final layout.

Table 5.1: Final circuit parameters. Generated in the same way as Tables 2.1, 3.1 and 4.1.

Parameter	signoff
Legality violations	0
Total overflow	58
Overflowing GRCs %	0.39
Utilization ratio	0.6713

Parameter	signoff
Chip area	2254.280
Core area	1385.2800
- Excluded area	0.0000
- Total cell area	930.0024
| - Combinational area	498.57
| - Noncombinational area	431.43
Power (pW)	6.88×10^7
- Dynamic power (pW)	6.85×10^7
| - Cell internal power (pW)	3.95×10^7
| - Net switching power (pW)	2.90×10^7
- Leakage power (pW)	3.21×10^5
Critical path length (ns)	1.73
Worst setup slack (ns)	−0.02
Worst hold slack (ns)	−0.00
Net length	11 802.76
Number of nets	1676
Number of cells	1252
- Number of buffers	37
- Number of inverters	23

Interesting details observed in logs and reports:

- The first `signoff_check_drc` listed 80 errors in total from 9 rules.
- The second one listed 32 errors from 7 rules.
- `signoff_create_metal_fill` increased density of layers M2-M5 from 5.9%-9.6% to 34.5%-44.9% and of layer M6 from 18% to 62%.
- `write_gds` command emitted a lot of warnings about unplaced cells beeing written to the GSDII file. The ones that were checked manually exist on layout and have attributes `is_placed true` and `physical_status placed`.

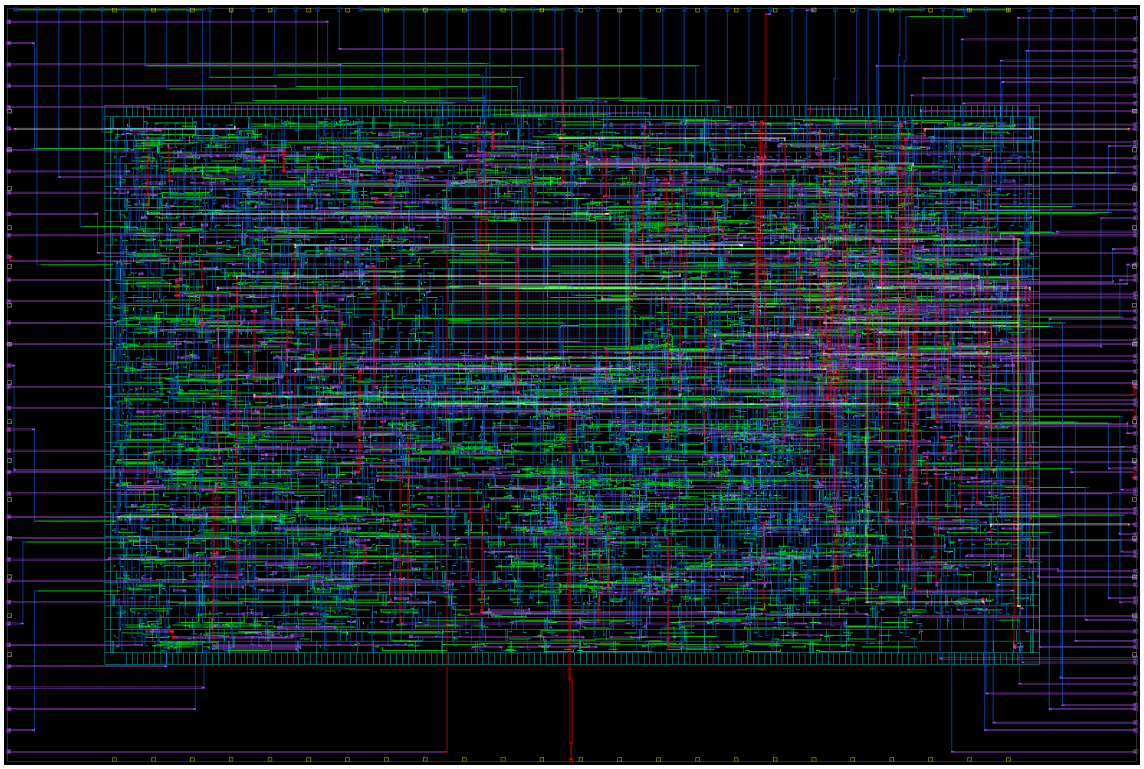

Figure 5.2: Final layout with power and ground routes hidden.